10.5281/zenodo.11526785

МАКСИМОВ Вячеслав Юрьевич
ведущий инженер-программист,
AUTO1 IT Services SE & Co. KG, Германия, г. Берлин

ПРОГРЕССИВНЫЙ ПАРАЛЛЕЛИЗМ НА ПРАКТИКЕ: ОТ КЛАССИЧЕСКИХ ПОТОКОВ К ВИРТУАЛЬНЫМ В JAVA И КОРУТИНАМ В KOTLIN

Аннотация. *Рассматривается использование многопоточности в современных вычислительных системах через призму Java и Kotlin на платформе JVM. Основное внимание уделено анализу эффективности традиционных потоков JVM и современных абстракций, таких как корутины в Kotlin и виртуальные потоки в Java. Статья изучает влияние этих технологий на производительность приложений, используя количественные методы для оценки задержек, потребления памяти и количества потоков в различных сценариях испытаний. Результаты тестов представляют интерес для определения наиболее эффективных подходов к параллелизму в рамках JVM, особенно в условиях высокой конкуренции за системные ресурсы.*

Ключевые слова: *Java, Kotlin, потоки, виртуальные потоки, корутины, структурированный параллелизм.*

Введение

В современных вычислительных системах параллельная обработка и многопоточность являются ключевыми элементами для улучшения производительности и эффективности приложений. JVM предлагает абстракции для создания и управления потоками в приложениях, которые напрямую взаимодействуют с потоками операционной системы (ОС) [1, с. 1-3]. Это взаимодействие критически важно для оптимизации приложений и эффективного использования системных ресурсов.

Основой для создания многопоточных приложений на Java является модель памяти Java (Java Memory Model, JMM), которая определяет, как изменения, внесенные одним потоком, могут быть видимы другими потоками. JMM определяет, как потоки в Java взаимодействуют с памятью [2, с. 81-96]. Она гарантирует, что изменения, сделанные одним потоком, будут видны другим, обеспечивая основу для безопасной многопоточности. JMM обеспечивает надлежащую синхронизацию с помощью таких конструкций, как «synchronized» блоки, «volatile» переменные и барьеры памяти. Это очень важно для предотвращения гонок данных и обеспечения согласованного поведения в многопоточных программах на Java.

- Стек (Stack) – Каждый поток, выполняющийся в виртуальной машине Java, имеет свой собственный стек потоков. Локальные переменные для примитивных типов полностью хранятся в стеке потоков и не видны другим потокам. Даже если два потока выполняют один и тот же код, они создадут свои отдельные копии локальных переменных для этого кода в своих стеках потоков. Память стека увеличивается при вызове новых методов и уменьшается при их завершении. Каждый раз, когда вызывается метод, создается новый кадр стека, и при возврате значения методом соответствующий кадр удаляется.
- Куча (Heap) – это отдельная область памяти, содержащая все объекты, созданные в приложении Java, независимо от того, какой поток их создал. Сюда входят объекты примитивных типов (например, Integer, Long). Независимо от того, был ли объект создан и присвоен локальной переменной или создан как переменная-член другого объекта, он сохраняется в куче.

Потоки в Java создаются и управляются через API класса java.lang.Thread, каждый из которых сопоставляется на поток операционной системы, позволяя JVM эффективно управлять выполнением кода на многопроцессорных системах. Потоки JVM полностью интегрированы

с планировщиком потоков ОС, что позволяет приложениям на Java воспользоваться преимуществами многопоточности на уровне аппаратного обеспечения. Поскольку потоки JVM являются потоками ОС, они подвержены политикам планирования ОС, что влияет на порядок и время выполнения потоков в приложении.

Kotlin использует те же механизмы Java Virtual Machine (JVM) для работы с потоками, что и Java, это позволяет использовать то же API классов по управлению жизненного цикла потока.

Отмечается ряд ограничений, с которыми сталкиваются JVM потоки. Частое создание и переключение потоков может привести к значительным накладным расходам, особенно на системах с большим количеством потоков, что снижает общую производительность приложения. Необходимо учитывать, поток JVM занимает определенное количество памяти и системных ресурсов, включая собственный стек вызовов и локальные переменные, что может привести к избыточному потреблению ресурсов при неоптимальном использовании потоков. Кроме того, потоки зависят от числа ядер процессора; создание большего числа потоков, чем количество ядер приведет к деградации производительности из-за конкуренции за процессорное время.

Появление легковесных потоков, представляющих собой абстракцию над традиционными потоками операционной системы, позволило уменьшить накладные расходы на создание и управление потоками. Они являются более масштабируемым и ресурсоэффективным способом для выполнения параллельных задач по сравнению со стандартными потоками, поскольку не требуют выделения отдельного стека и других системных ресурсов для каждого потока. Легкие потоки не управляются напрямую операционной системой, а организуются на уровне пользовательского пространства или библиотеки. Это снижает время и ресурсы, необходимые для создания и переключения контекстов. Также они позволяют создавать большое количество параллельных задач, не перегружая систему, благодаря эффективному распределению задач между физическими потоками.

Реализация легких потоков существует в ряде языков и имеет различные термины, например в Kotlin это Coroutines, в Go – Goroutines, в Erlang – Processes и т. д. В недавней 21 версии Java появилась официальная поддержка виртуальных потоков (Virtual Threads).

Корутины в Kotlin

Поддержка легких потоков в Kotlin осуществляется через корутины [3, с. 68-84], реализованные в виде богатой библиотеки kotlinx.coroutines. Корутины перешли из статуса экспериментального в стабильной версии Kotlin 1.3. Они не привязаны к потокам ядра ОС, coroutine позволяет выполнение задач в одном или нескольких потоков JVM.

В отличие от традиционных функций, которые выполняются от начала до конца без остановки, корутины могут «приостанавливаться» в точках ожидания (где выполняется асинхронная операция) и «возобновляться» с того же места, как только асинхронная операция завершится. Это достигается благодаря использованию специальных «приостанавливающих функций» (suspend functions), которые отмечают точки, где корутина может быть приостановлена [4, с. 27-38]. Kotlin предоставляет языковую поддержку и библиотеки, такие как kotlinx.coroutines, для управления жизненным циклом корутин и выполнения асинхронных задач.

Корутины в Kotlin запускаются в определенной области видимости (CoroutineScope), а для запуска самой корутины используется launch, где запуск возвращает задание и не несет никакого результирующего значения или async возвращающий отложенное задание Deffered.

Корутины всегда выполняются в некотором контексте, представленном значением типа CoroutineContext, определенного в стандартной библиотеке Kotlin. Контекст корутины включает в себя диспетчер сопрограммы (CoroutineDispatcher), который определяет, какой поток или потоки соответствующая корутина использует для своего выполнения. Диспетчер сопрограммы может ограничить выполнение корутины определенным потоком, отправить ее в пул потоков или позволить ей выполняться без ограничений.

Существует 4 вида диспетчеров [5]:

1. Dispatchers.Default: по умолчанию используется всеми стандартными сборщиками, такими как запуск, асинхронность и т. д., если в их контексте не указан диспетчер;
2. Dispatchers.IO: предназначенный для блокирующих задач ввода-вывода;
3. Dispatchers.Main: ограниченный основным потоком, работающим с объектами пользовательского интерфейса;

4. Dispatchers.Unconfined: не привязанный к какому-либо конкретному потоку. Он выполняет начальное продолжение корутины в текущем кадре вызова и позволяет корутине возобновить работу в любом потоке, который используется соответствующей приостанавливающей функцией, без указания какой-либо конкретной политики потоков.

CoroutineScope управляет жизненным циклом корутин в рамках этой области, обеспечивая автоматическую отмену всех запущенных в данной области корутин, когда область видимости уничтожается или завершается, предотвращая таким образом утечки памяти и ресурсов.

Структурированный параллелизм в корутинах позволяет локализовать и контролировать жизненные циклы корутин, уменьшая риск утечек ресурсов и других связанных с асинхронностью проблем. Корутины, запущенные в рамках одной области видимости, могут быть легко синхронизированы и координированы, это помогает корректно реагировать на ошибки и отмены, автоматически прекращать все связанные операции, минимизируя возможные побочные эффекты ошибок или нежелательных состояний.

Виртуальные потоки (Virtual threads) в Java

В версии 21 языка Java стали официально доступны виртуальные потоки [6]. Они управляются JVM и предназначены для решения недостатков традиционных потоков операционной системы (OS threads), таких, как накладные расходы, связанные с ресурсами и планированием.

Программирование с виртуальными потоками похоже на использование обычных потоков. Виртуальные потоки в Java предназначены для использования с существующими интерфейсами и классами в стандартной библиотеке Java, такими как Runnable, Callable и ExecutorService. Это позволяет легко адаптировать существующий многопоточный код для использования виртуальных потоков. Для запуска виртуального потока достаточно использовать расширенный метод startVirtualThread из java.lang.Thread [7].

Виртуальные потоки в Java реализуют модель «многие-ко-многим», где множество виртуальных потоков могут быть отображены на относительно меньшее количество физических потоков (потоки операционной системы). Когда виртуальный поток ожидает ввода/вывода или блокировки, он может быть приостановлен, и его физический поток может быть использован для выполнения других виртуальных потоков. Очевидно, что виртуальные потоки служат для приложений с высокими требованиями к параллелизму и асинхронности, достигая лучшую масштабируемость и более быструю синхронизацию.

Необходимо использовать виртуальные потоки в параллельных приложениях с высокой пропускной способностью, особенно в тех, которые состоят из большого количества одновременных задач и тратящие значительную часть времени на ожидание. Серверные приложения являются примерами приложений с высокой пропускной способностью, поскольку они обычно обрабатывают множество клиентских запросов, которые выполняют блокирующие операции ввода-вывода, такие, как выборка ресурсов.

Тестовая среда и методология

Все тесты выполнялись на одном компьютере модели MacBook Pro (16-inch, 2021), состоящем из 64-битной машины с операционной системой macOS Monterey 12.6, 32 ГБ памяти типа LPDDR5 и процессором 3.2 GHz Apple M1 Pro. Для запуска JVM использовался arch-64 OpenJDK-21.0.2 LTS, версия языка Kotlin 1.9.22.

Одна и та же программа была написана на Java и Kotlin с использованием различных конструкций параллелизма. Всего было 3 тестовых случая – для Java с использованием обычных потоков и виртуальных потоков, и для Kotlin с использованием корутин. Каждый тест представляет собой работу с диском путем записи в файл информации о дате создания для каждого потока. Coroutines использует контекст Dispatchers.IO. Dispachers.IO по умолчанию устанавливает ограничение в 64 потока или количество ядер, в зависимости от того, что больше [8]. В Java – количество потоков платформы, доступных для планирования виртуальных потоков по умолчанию используется количество доступных процессоров [9].

Для определения сбалансированного среднего значения производилось 10 последовательных запросов, в которых создавались стандартные потоки, виртуальные потоки или корутины на 10, 100, 1000 или 10000 повторений, что позволило получить представление о масштабируемости и эффективности каждого инструмента конкурентности при увеличении нагрузки. Для просмотра результатов по тестам использовался инструмент VisualVM 2.1.7,

обеспечивающий визуальное представление информации, в том числе о потоках, CPU и памяти для JVM приложений.

Результаты и анализ

При тестировании средней задержки для классических потоков, виртуальных потоков и корутин (табл. 1), выявилось, что на низких уровнях (10 и 100) все три случая показали близкую производительность с небольшими вариациями. Значительные различия в производительности проявились на высоких уровнях (1000 и 10000). Java Virtual Threads систематически показывали лучшую производительность по сравнению с Java Threads, существенно снижая задержку в 3 раза на наивысшем испытанном уровне. Kotlin Coroutines также превосходили Java Threads. Примечательно, виртуальные потоки и корутины работают почти одинаково относительно производительности, но с небольшим преимуществом на стороне виртуальных потоков.

Таблица 1

Средняя задержка в миллисекундах в зависимости от количества повторений

Repetitions	Java Threads	Java Virtual Threads	Kotlin Coroutines
10	16	22	21
100	25	23	32
1000	80	38	53
10000	537	152	165

Рассмотрим среднее потребление памяти «кучи» при максимальной нагрузке (табл. 2). Видно, что виртуальные потоки и корутины работают значительно лучше, чем обычные потоки в Java, так как занимают заметно меньше места в куче. Поскольку потоки являются оберткой для потоков ядра ОС, их сложнее удалить из кучи, что в конечном итоге приводит к «засорению» кучи. Значительное количество потоков ядра ОС используется для обработки высокой частоты запросов, они занимают ресурсы, необходимые для эффективного управления памятью и сборки мусора. По сравнению с потоками, в основном благодаря лучшим планировщикам, и виртуальные потоки, и корутины освобождают свой поток ядра ОС из кучи, как только они больше не нужны, без лишнего удержания.

Таблица 2

Среднее потребление «кучи» в мегабайтах для максимально испытанного количества повторений

Repetitions	Java Threads	Java Virtual Threads	Kotlin Coroutines
10000	640	12	11

Как обсуждалось ранее, виртуальные потоки и корутины предусматривают меньшее количество действительных потоков (табл. 3). При низкой нагрузке (10 и 100) разница в количестве потоков между Java Virtual Threads и Kotlin Coroutines невелика, что указывает на почти одинаковое использование ресурсов. Однако при высокой нагрузки (1000 и 10000) разница становится значительной. Coroutines потребляют почти в два раза больше потоков, чем Virtual Threads, что может указывать на более высокую ресурсоемкость.

Таблица 3

Среднее количество созданных потоков для 10, 100, 1000 и 10000 повторений

Repetitions	Java Virtual Threads	Kotlin Coroutines
10	11	13
100	18	20
1000	20	39
10000	40	77

Обсуждения

Одной из основных целей параллелизма является оптимизация использования многоядерных процессоров и повышение производительности приложений при том же количестве ресурсов. Однако появление новых подходов к параллелизму, включая разработку новых реализаций, таких как виртуальные потоки, может принести и другие преимущества.

Сравнивая языки программирования .Net C# и Java, известно несмотря на то, что язык программирования Java предназначен для

работы на нескольких платформах, что усложняет проектирование и реализацию такой среды, он является значительно быстрее [10, с. 426-441]. Также согласно сравнительному анализу Java, Python and GO [11], Java и Go соревнуются между собой и занимают лидирующие позиции в зависимости от характера использования в многопоточной среде. Более того, современные полиглотские виртуальные машины позволяют разработчикам использовать преимущества полиглотского сочетания и синергии различных языков программирования. Одним из таких примеров является виртуальная машина (ВМ) GraalVM, которая поддерживает разные языки программирования и модели выполнения, такие как JIT-компиляция и AOT-компиляция. Это означает, что дальнейшее развитие ВМ и добавление новых языков, таких как Kotlin, может привнести преимущества структурированного параллелизма в полиглот-программирование, позволяя разработчикам комбинировать различные языки программирования, такие как Java и Kotlin, для достижения высокой производительности более простым способом.

По результатам тестов, поскольку виртуальные потоки и корутины освобождают свои потоки ядра ОС значительно быстрее, они могут улучшить процесс управления памятью. Все потоки JVM имеют доступ к куче, улучшение скорости синхронизации между потоками приводит к лучшему управлению объектами, хранящимися в куче, и их удалению с помощью GC. В Java 21 по прежнему по умолчанию используется сборщик мусора (GC) G1, и для дальнейшего исследования следует проверить различные сборщики мусора, например Shenandoah GC и Generational ZGC с виртуальными потоками. В частности, изучению различий между поведением и производительностью виртуальных потоков и корутинов в этом отношении. Расширенный набор тест кейсов для работы с сетью или исключая работу с блокирующими задачами ввода/вывода предоставят более детальные варианты использования того или иного подхода.

Заключение

Исследование подтвердило значительные преимущества использования легковесных потоков, таких как виртуальные потоки и корутины, по сравнению с традиционными потоками JVM в контексте мультипотокового программирования. Виртуальные потоки Java и корутины Kotlin демонстрируют улучшенную производительность на высоких уровнях нагрузки благодаря эффективному управлению памятью и меньшим затратам на синхронизацию. Эти технологии способствуют более высокой пропускной способности и лучшему распределению задач, особенно в многозадачных и асинхронных приложениях. Также, структурированный параллелизм корутин в Kotlin и механизмы управления виртуальными потоками в Java предлагают значительные улучшения для приложений, требующих интенсивной работы с потоками. Отмечается схожая производительность виртуальных потоков и корутин, однако виртуальные потоки превосходили корутины по скорости выполнения и меньшему потреблению ресурсов процессора. Однако выбор одного из предоставленных подходов по применению легковесных потоков зависит от варианта использования при построении системы и узконаправленных анализов производительности.

Литература

1. Goetz B. Java Concurrency in Practice / Goetz B: Addison-Wesley, 2010 – 403 с.

2. Huisman M., Petri G. The Java memory model: a formal explanation // VAMP. – 2007. – Т. 7. – С. 81-96.

3. Kotlin coroutines: Design and implementation / Elizarov R., Belyaev M., Akhin M., Usmanov I. // Onward! 2021 – Proceedings of the 2021 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, co-located with SPLASH 2021, Virtual, Online, 20–22 октября 2021 года. – Virtual, Online, 2021. – P. 68-84. – DOI 10.1145/3486607.3486751. – EDN HMTVVU. (Elizarov R., Belyaev M., Akhin M., Usmanov I., "Kotlin Coroutines: Design and Implementation", Proc. of the 2021 ACM SIGPLAN Int. Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!' 21), Chicago, IL, USA, ACM, New York, NY, USA, 2021, p. 68-84, doi: 10.1145/3486607.3486751).

4. Moskała M, Kotlin Coroutines: Deep Dive / Moskała M – 2., 2023 – 463 с.

5. Kotlin Coroutines dispatchers. // Kt. Academy. URL: https://kt.academy/article/cc-dispatchers (дата обращения: 01.06.2024).

6. Virtual Threads. // Docs Oracle. URL: https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html (дата обращения 19.05.2024).

7. Going inside Java's Project Loom and virtual threads. // Java Magazine. URL: https://blogs.oracle.com/javamagazine/post/going-inside-javas-project-loom-and-virtual-threads (дата обращения: 01.06.2024).

8. Dispatchers.IO. // Kotlinlang. URL: https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines/-dispatchers/-i-o.html (дата обращения 19.05.2024).

9. Class Thread. // Docs Oracle. URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html (дата обращения 19.05.2024).

10. Zentner A. Multithreading in .Net and Java: A Reality Check / Zentner A. // Journal of Computers. – 2018. С. 426-441.

11. Java, Go, and Python: a multi-thread performance comparison. // Level up coding. URL: https://levelup.gitconnected.com/java-go-and-python-a-multi-thread-performance-comparison-28e942cb73e6 (дата обращения 19.05.2024).

MAKSIMOV Viacheslav Yurievich
Senior Software Engineer,
AUTO1 IT Services SE & Co. KG, Germany, Berlin

PROGRESSIVE PARALLELISM IN PRACTICE: FROM CLASSICAL THREADS TO VIRTUAL THREADS IN JAVA AND COROUTINES IN KOTLIN

Abstract. *We consider the use of multithreading in modern computing systems through the prism of Java and Kotlin on the JVM platform. The focus is on analyzing the performance of traditional JVM threads and modern abstractions such as coroutines in Kotlin and virtual threads in Java. The paper investigates the impact of these technologies on application performance, using quantitative techniques to evaluate latency, memory consumption, and thread counts in different test scenarios. The test results are of interest in determining the most effective approaches to parallelism within the JVM, especially in a highly competitive environment for system resources.*

Keywords: *Java, Kotlin, threads, Virtual threads, Coroutines, structured concurrency.*